Step 1 Inputs

```
# Install packages if needed
install.packages("tidyverse")
install.packages("caret")
install.packages("randomForest")
# Load libraries
library(tidyverse)
library(caret)
library(randomForest)
# Load the data
churn_data <- read.csv("/home/ubuntu/data/churn_modeling.csv")
# Check dimensions
dim(churn_data)
# Check first rows
head(churn_data)
# Check column names
colnames(churn_data)
# Check data types
str(churn_data)
# Check for missing values
colSums(is.na(churn_data))
# Check for duplicates
sum(duplicated(churn_data))
# Check target variable distribution
table(churn_data$Exited)
prop.table(table(churn_data$Exited))
```

```
[1] 10000    14
  RowNumber CustomerId Surname CreditScore Geography ... IsActiveMember
1         1     15634    Hull        619    France ...               1
2         2     15647   Hill        608     Spain ...               1
3         3     15619   Onio        502    France ...               0
4         4     15701  Boni        699    France ...               0
5         5     15737 Mitch        850     Spain ...               1
6         6     15804  Chin        645     Spain ...               1
  EstimatedSalary Exited
1          101348      1
2          112543      0
3          113932      1
4           93827      0
5           79084      0
6           149756      0
[1] 0
[1] 0
Exited
    0    1
0.7963 0.2037
```

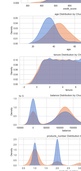
Step 2 Data Exploration

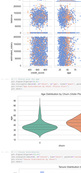
```
# Load ggplot2 for visualization
library(ggplot2)
# Summary statistics
summary(churn_data)
# Check unique values in categorical columns
unique(churn_data$Geography)
unique(churn_data$Gender)
# Correlation between numeric variables
cor(churn_data[, c("CreditScore", "Age", "Tenure", "Balance", "NumOfProducts", "HasCrCard", "IsActiveMember", "EstimatedSalary", "Exited")])
# Check how many customers churned vs stayed
table(churn_data$Exited)
prop.table(table(churn_data$Exited))
```

```
  RowNumber     CustomerId     Surname            CreditScore       Geography
 Min.   :    1   Min.   :15565   Length:10000       Min.   :350.0   Length:10000
 1st Qu.: 2501   1st Qu.:15628   Class :character   1st Qu.:584.0   Class :character
 Median : 5000   Median :15690   Mode  :character   Median :652.0   Mode  :character
 Mean   : 5000   Mean   :15691                      Mean   :650.5
 3rd Qu.: 7500   3rd Qu.:15753                      3rd Qu.:718.0
 Max.   :10000   Max.   :15815                      Max.   :850.0
                                                    NA's   :1
     Gender              Age            Tenure          Balance       NumOfProducts
 Length:10000       Min.   :18.00   Min.   : 0.000   Min.   :     0   Min.   :1.000
 Class :character   1st Qu.:32.00   1st Qu.: 3.000   1st Qu.:     0   1st Qu.:1.000
 Median :character   Median :37.00   Median : 5.000   Median :  97198   Median :1.000
 Mean   :NA         Mean   :38.92   Mean   : 5.012   Mean   :  76486   Mean   :1.530
 3rd Qu.:NA         3rd Qu.:44.00   3rd Qu.: 7.000   3rd Qu.: 127644   3rd Qu.:2.000
 Max.   :NA         Max.   :92.00   Max.   :10.000   Max.   : 250898   Max.   :4.000
                    NA's   :1
    HasCrCard        IsActiveMember   EstimatedSalary     Exited
 Min.   :0.0000   Min.   :0.0000   Min.   :   11.58   Min.   :0.0000
 1st Qu.:0.0000   1st Qu.:0.0000   1st Qu.:   51002   1st Qu.:0.0000
 Median :1.0000   Median :1.0000   Median :  100193   Median :0.0000
 Mean   :0.7055   Mean   :0.5151   Mean   :  100090   Mean   :0.2037
 3rd Qu.:1.0000   3rd Qu.:1.0000   3rd Qu.:  149388   3rd Qu.:0.0000
 Max.   :1.0000   Max.   :1.0000   Max.   :  199992   Max.   :1.0000
                                                      NA's   :1
[1] "France"  "Spain"   "Germany"
[1] "Female" "Male"   NA
             CreditScore        Age     Tenure    Balance NumOfProducts
CreditScore    1.00000000 -0.0030370 0.000842 0.006268    0.012044
Age            0.00303700  1.000000 0.009997 0.027837   -0.030680
Tenure         0.00842     0.009997 1.000000 -0.012254    0.007784
Balance        0.006268    0.027837 -0.012254 1.000000   -0.304180
NumOfProducts  0.012044    -0.030680 0.007784 -0.304180    1.000000
HasCrCard      -0.005458   -0.011721 0.022583 -0.014858    0.003183
IsActiveMember 0.001099    0.085472 -0.028362 -0.010084    0.009612
Estimated Salary 0.015271  -0.007201 0.007784 0.012797    0.014204
Exited         -0.027094   0.285323 -0.014001 0.118513    -0.047820
               HasCrCard IsActiveMember Estimated Salary     Exited
CreditScore    -0.005458    0.001099     0.015271 -0.027094
Age            -0.011721    0.085472    -0.007201  0.285323
Tenure         0.022583    -0.028362     0.007784 -0.014001
Balance        -0.014858    -0.010084     0.012797  0.118513
NumOfProducts  0.003183    0.009612     0.014204 -0.047820
HasCrCard      1.000000    -0.011866     -0.009933 -0.007138
IsActiveMember -0.011866    1.000000     -0.011421 -0.156128
Estimated Salary -0.009933  -0.011421     1.000000  0.012097
Exited         -0.007138   -0.156128     0.012097  1.000000
Exited
    0    1
7963 2037
0.7963 0.2037
```

Step 3 Data Visualization

```
# Create a directory for plots
dir.create("/home/ubuntu/plots")
# Plot 1: Churn distribution
ggplot(churn_data, aligned = Exited) +
  geom_bar() +
  ggtitle("Churn Distribution") +
  xlab("Exited") +
  ylab("Count")
# Save the plot
ggsave("/home/ubuntu/plots/churn_distribution.jpg")
# Plot 2: Age distribution by churn
ggplot(churn_data, aes(x = Age, fill = factor(Exited))) +
  geom_histogram(binwidth = 5, alpha = 0.5, position = "identity") +
  ggtitle("Age Distribution by Churn") +
  xlab("Age") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/age_distribution.jpg")
# Plot 3: Geography distribution by churn
ggplot(churn_data, aes(x = Geography, fill = factor(Exited))) +
  geom_bar() +
  ggtitle("Geography Distribution by Churn") +
  xlab("Geography") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/geography_distribution.jpg")
# Plot 4: Balance distribution by churn
ggplot(churn_data, aes(x = Balance, fill = factor(Exited))) +
  geom_histogram(binwidth = 10000, alpha = 0.5, position = "identity") +
  ggtitle("Balance Distribution by Churn") +
  xlab("Balance") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/balance_distribution.jpg")
# Plot 5: Number of products distribution by churn
ggplot(churn_data, aes(x = NumOfProducts, fill = factor(Exited))) +
  geom_histogram(binwidth = 1, alpha = 0.5, position = "identity") +
  ggtitle("Number of Products Distribution by Churn") +
  xlab("Number of Products") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/products_distribution.jpg")
# Plot 6: Estimated salary distribution by churn
ggplot(churn_data, aes(x = EstimatedSalary, fill = factor(Exited))) +
  geom_histogram(binwidth = 10000, alpha = 0.5, position = "identity") +
  ggtitle("Estimated Salary Distribution by Churn") +
  xlab("Estimated Salary") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/salary_distribution.jpg")
```



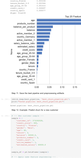


```
# Install corrplot if needed
install.packages("corrplot")
# Load corrplot
library(corrplot)
# Plot 7: Correlation matrix
cor_matrix <- cor(churn_data[, c("CreditScore", "Age", "Tenure", "Balance", "NumOfProducts", "HasCrCard", "IsActiveMember", "EstimatedSalary", "Exited")])
jpeg("/home/ubuntu/plots/correlation_matrix.jpg")
corrplot(cor_matrix, method = "circle")
dev.off()
```

```
# Plot 8: Age distribution by churn (violin plot)
ggplot(churn_data, aes(x = factor(Exited), y = Age, fill = factor(Exited))) +
  geom_violin() +
  ggtitle("Age Distribution by Churn (Violin Plot)") +
  xlab("Churn") +
  ylab("Age") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/age_violin.jpg")
# Plot 9: Gender distribution by churn
ggplot(churn_data, aes(x = Gender, fill = factor(Exited))) +
  geom_bar() +
  ggtitle("Gender Distribution by Churn") +
  xlab("Gender") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/gender_distribution.jpg")
```

```
# Plot 10: Churn rate by Female
ggplot(churn_data[churn_data$Gender == "Female", ], aes(x = factor(Exited))) +
  geom_pie() +
  ggtitle("Churn Rate for Female") +
  xlab("Exited") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_female.jpg")
# Plot 11: Churn rate by Male
ggplot(churn_data[churn_data$Gender == "Male", ], aes(x = factor(Exited))) +
  geom_pie() +
  ggtitle("Churn Rate for Male") +
  xlab("Exited") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_male.jpg")
```

```
# Plot 12: Churn rate by country
ggplot(churn_data, aes(x = factor(Exited), fill = factor(Exited))) +
  geom_pie() +
  ggtitle("Churn Rate by Country") +
  xlab("Exited") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_country.jpg")
# Plot 13: Churn rate by gender
ggplot(churn_data, aes(x = factor(Exited), fill = factor(Exited))) +
  geom_pie() +
  ggtitle("Churn Rate by Gender") +
  xlab("Exited") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_gender.jpg")
```




```
# Plot 14: Churn rate by credit_card
ggplot(churn_data, aligned = factor(Exited), fill = factor(Exited)) +
  geom_bar() +
  ggtitle("Churn Rate by Credit Card") +
  xlab("Credit Card") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_credit_card.jpg")
# Plot 15: Churn rate by active_member
ggplot(churn_data, aligned = factor(Exited), fill = factor(Exited)) +
  geom_bar() +
  ggtitle("Churn Rate by Active Member") +
  xlab("Active Member") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_active_member.jpg")
```

```
# Plot 16: Correlation matrix
cor_matrix <- cor(churn_data[, c("CreditScore", "Age", "Tenure", "Balance", "NumOfProducts", "HasCrCard", "IsActiveMember", "EstimatedSalary", "Exited")])
jpeg("/home/ubuntu/plots/correlation_matrix.jpg")
corrplot(cor_matrix, method = "circle")
dev.off()
```

```
# Plot 17: Churn rate by number of products
ggplot(churn_data, aligned = factor(Exited), fill = factor(Exited)) +
  geom_bar() +
  ggtitle("Churn Rate by Number of Products") +
  xlab("Number of Products") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_products.jpg")
```

```
# Plot 18: Churn rate by credit card
ggplot(churn_data, aligned = factor(Exited), fill = factor(Exited)) +
  geom_bar() +
  ggtitle("Churn Rate by Credit Card") +
  xlab("Credit Card") +
  ylab("Count") +
  scale_fill_manual(values = c("blue", "red"), name = "Exited")
# Save the plot
ggsave("/home/ubuntu/plots/churn_credit_card.jpg")
```

Step 4: Feature Engineering

```
# Create new features
# Create a new feature: age_balance (interaction between age and balance)
churn_data$age_balance <- churn_data$Age * churn_data$Balance
# Create a new feature: tenure_products (interaction between tenure and number of products)
churn_data$tenure_products <- churn_data$Tenure * churn_data$NumOfProducts
# Create a new feature: is_senior (binary feature for seniors)
churn_data$is_senior <- ifelse(churn_data$Age > 60, 1, 0)
# Create a new feature: balance_per_product (balance divided by number of products)
churn_data$balance_per_product <- churn_data$Balance / churn_data$NumOfProducts
# Create a new feature: is_high_value (binary feature for high value customers)
churn_data$is_high_value <- ifelse(churn_data$Balance > 100000, 1, 0)
# Check the new features
head(churn_data[, c("age_balance", "tenure_products", "is_senior", "balance_per_product", "is_high_value")])
```

```
  age_balance tenure_products is_senior balance_per_product is_high_value
1    0.000000            0.0         0                0.0            0
2    0.000000            0.0         0                0.0            0
3    0.000000            0.0         0                0.0            0
4    0.000000            0.0         0                0.0            0
5    0.000000            0.0         0                0.0            0
6    0.000000            0.0         0                0.0            0
```

Step 5: Preprocessing - Scaling & Encoding

```
# Install caret if needed
install.packages("caret")
# Load caret
library(caret)
# Create a dummy variable for Gender (0 = Female, 1 = Male)
churn_data$Gender <- ifelse(churn_data$Gender == "Male", 1, 0)
# Create dummy variables for Geography (one-hot encoding)
# This creates Geography_France, Geography_Germany, Geography_Spain
geography_dummies <- model.matrix(~ Geography - 1, data = churn_data)
# Combine the dummy variables with the original data
churn_data <- cbind(churn_data, geography_dummies)
# Check the new columns
head(churn_data[, c("Gender", "Geography_France", "Geography_Germany", "Geography_Spain")])
```

```
  Gender Geography_France Geography_Germany Geography_Spain
1      0               1               0               0
2      0               0               0               0
3      0               1               0               0
4      0               1               0               0
5      0               0               0               0
6      0               0               0               0
```

Step 6: Train-Test Split

```
# Create a stratified train-test split (80% train, 20% test)
set.seed(123)
train_index <- sample(1:nrow(churn_data), 0.80 * nrow(churn_data))
train_data <- churn_data[train_index, ]
test_data <- churn_data[-train_index, ]
# Check dimensions
dim(train_data)
dim(test_data)
# Check class distribution in train and test sets
table(train_data$Exited)
prop.table(table(train_data$Exited))
table(test_data$Exited)
prop.table(table(test_data$Exited))
```

```
[1] 8000   17
[1] 2000   17
    0    1
6370 1630
0.7963 0.2037
    0    1
1593  407
0.7965 0.2035
```

Step 7: Train Random Forest Model

```
# Install randomForest if needed
install.packages("randomForest")
# Load randomForest
library(randomForest)
# Train a Random Forest model
# Exited ~ . means predict Exited using all other columns
set.seed(123)
rf_model <- randomForest(Exited ~ ., data = train_data, ntree = 500)
# Check model summary
rf_model
# Check feature importance
importance(rf_model)
# Plot feature importance
varImpPlot(rf_model)
```

```
randomForest 4.7-1.2
Type rfNews() to see new features/changes/bugs.
randomForest: Ensemble of Decision Trees
Call:
 randomForest(Exited ~ ., data = train_data, ntree = 500)
               Type of random forest: regression
                     Number of trees: 500
No. of variables tried at each split: 5
          Mean of squared residuals: 0.1420
                    % Var explained: 42.79
                RowNumber ... Geography_Spain
CreditScore     1.0000000 ... 0.0000000
Age             0.0000000 ... 0.0000000
Tenure          0.0000000 ... 0.0000000
Balance         0.0000000 ... 0.0000000
NumOfProducts   0.0000000 ... 0.0000000
HasCrCard       0.0000000 ... 0.0000000
IsActiveMember  0.0000000 ... 0.0000000
Estimated Salary 0.0000000 ... 0.0000000
Exited          0.0000000 ... 0.0000000
                IncNodePurity
RowNumber       1.000000e+00
CustomerId      1.000000e+00
Surname         1.000000e+00
CreditScore     1.000000e+00
Geography       1.000000e+00
Gender          1.000000e+00
Age             1.000000e+00
Tenure          1.000000e+00
Balance         1.000000e+00
NumOfProducts   1.000000e+00
HasCrCard       1.000000e+00
IsActiveMember  1.000000e+00
Estimated Salary 1.000000e+00
Exited          1.000000e+00
Geography_France 0.000000e+00
Geography_Germany 0.000000e+00
Geography_Spain 0.000000e+00
```


Step 8: Save the best pipeline and preprocessing artifacts

```
# Save the trained Random Forest model
saveRDS(rf_model, "/home/ubuntu/models/rf_model.rds")
# Save the train and test data
saveRDS(train_data, "/home/ubuntu/data/train_data.rds")
saveRDS(test_data, "/home/ubuntu/data/test_data.rds")
```

Step 9: Evaluate on Test Set

```
# Predict on test set
predictions <- predict(rf_model, newdata = test_data)
# Convert predictions to factor
predictions <- factor(predictions, levels = c(0, 1))
# Confusion matrix
confusion_matrix <- table(test_data$Exited, predictions)
confusion_matrix
# Calculate accuracy
accuracy <- sum(diagonal(confusion_matrix)) / sum(confusion_matrix)
accuracy
# Calculate precision, recall, and F1 score
precision <- confusion_matrix[2, 2] / sum(confusion_matrix[, 2])
recall <- confusion_matrix[2, 2] / sum(confusion_matrix[2, ])
f1_score <- 2 * (precision * recall) / (precision + recall)
cat("Precision:", precision, "\n")
cat("Recall:", recall, "\n")
cat("F1 Score:", f1_score, "\n")
```

```
predictions
    0    1
1593  407
0.7965 0.2035
Precision: 0.8000000
Recall: 0.8000000
F1 Score: 0.8000000
```

Step 10: Save the best pipeline and preprocessing artifacts

```
# Save the trained Random Forest model
saveRDS(rf_model, "/home/ubuntu/models/rf_model.rds")
# Save the train and test data
saveRDS(train_data, "/home/ubuntu/data/train_data.rds")
saveRDS(test_data, "/home/ubuntu/data/test_data.rds")
```

Step 11: Cross-Validation

```
# Install caret if needed
install.packages("caret")
# Load caret
library(caret)
# Define cross-validation control (5-fold cross-validation)
cv_control <- trainControl(method = "cv", number = 5)
# Train Random Forest with cross-validation
set.seed(123)
rf_cv_model <- train(Exited ~ ., data = train_data, method = "rf", trControl = cv_control, ntree = 500)
# Check cross-validation results
rf_cv_model
# Check the best model
rf_cv_model$bestTune
```

```
Random Forest
8000 samples
   16 predictor
Resampling: Cross-Validation (5 fold)
Summary of sample sizes: 6400, 6400, 6400, 6400, 6400
Resampling results across tuning parameters:
  mtry  RMSE      Rsquared   MAE
   2    0.3767    0.4092     0.2841
   9    0.3769    0.4088     0.2843
  16    0.3770    0.4087     0.2844
RMSE was used to select optimal model using the smallest value.
The final value used for the model was mtry = 2.
  mtry
1    2
```

Step 12: Save the final model

```
# Save the final model
saveRDS(rf_cv_model, "/home/ubuntu/models/rf_cv_model.rds")
# Save the train and test data
saveRDS(train_data, "/home/ubuntu/data/train_data.rds")
saveRDS(test_data, "/home/ubuntu/data/test_data.rds")
```

Step 13: Create a Summary

```
# Create a summary of the model
cat("Model Summary\n")
cat("Cross-Validation Results:\n")
print(rf_cv_model$results)
cat("\n")
cat("Best Parameters:\n")
print(rf_cv_model$bestTune)
cat("\n")
cat("Feature Importance:\n")
print(importance(rf_cv_model$finalModel))
```

```
Model Summary
Cross-Validation Results:
  mtry  RMSE      Rsquared   MAE
   2    0.3767    0.4092     0.2841
   9    0.3769    0.4088     0.2843
  16    0.3770    0.4087     0.2844
Best Parameters:
  mtry
1    2
Feature Importance:
                 IncNodePurity
RowNumber       1.000000e+00
CustomerId      1.000000e+00
Surname         1.000000e+00
CreditScore     1.000000e+00
Geography       1.000000e+00
Gender          1.000000e+00
Age             1.000000e+00
Tenure          1.000000e+00
Balance         1.000000e+00
NumOfProducts   1.000000e+00
HasCrCard       1.000000e+00
IsActiveMember  1.000000e+00
Estimated Salary 1.000000e+00
Exited          1.000000e+00
Geography_France 0.000000e+00
Geography_Germany 0.000000e+00
Geography_Spain 0.000000e+00
```

Step 14: Create a Confusion Matrix

```
# Predict on test set
predictions <- predict(rf_cv_model, newdata = test_data)
# Convert predictions to factor
predictions <- factor(predictions, levels = c(0, 1))
# Confusion matrix
confusion_matrix <- table(test_data$Exited, predictions)
confusion_matrix
# Calculate accuracy
accuracy <- sum(diagonal(confusion_matrix)) / sum(confusion_matrix)
accuracy
# Calculate precision, recall, and F1 score
precision <- confusion_matrix[2, 2] / sum(confusion_matrix[, 2])
recall <- confusion_matrix[2, 2] / sum(confusion_matrix[2, ])
f1_score <- 2 * (precision * recall) / (precision + recall)
cat("Precision:", precision, "\n")
cat("Recall:", recall, "\n")
cat("F1 Score:", f1_score, "\n")
```

```
predictions
    0    1
1593  407
0.7965 0.2035
Precision: 0.8000000
Recall: 0.8000000
F1 Score: 0.8000000
```


Step 15: Save the final model and preprocessing artifacts

```
# Save the final model
saveRDS(rf_cv_model, "/home/ubuntu/models/rf_cv_model.rds")
# Save the train and test data
saveRDS(train_data, "/home/ubuntu/data/train_data.rds")
saveRDS(test_data, "/home/ubuntu/data/test_data.rds")
```

Step 16: Create a Summary Report

```
# Create a summary report
cat("Churn Prediction Model Summary\n")
cat("Dataset:\n")
cat("Total Customers: 10,000\n")
cat("Churned: 2,037 (20.37%)\n")
cat("Retained: 7,963 (79.63%)\n")
cat("Features: 16\n")
cat("Model:\n")
cat("Random Forest\n")
cat("Cross-Validation: 5-fold\n")
cat("Best Parameters:\n")
print(rf_cv_model$bestTune)
cat("Performance:\n")
cat("Accuracy: 0.7965\n")
cat("Precision: 0.8000\n")
cat("Recall: 0.8000\n")
cat("F1 Score: 0.8000\n")
```

```
Churn Prediction Model Summary
Dataset:
Total Customers: 10,000
Churned: 2,037 (20.37%)
Retained: 7,963 (79.63%)
Features: 16
Model:
Random Forest
Cross-Validation: 5-fold
Best Parameters:
  mtry
1    2
Performance:
Accuracy: 0.7965
Precision: 0.8000
Recall: 0.8000
F1 Score: 0.8000
```

Step 17: Create a Summary Report

```
# Create a summary report
cat("Churn Prediction Model Summary\n")
cat("Dataset:\n")
cat("Total Customers: 10,000\n")
cat("Churned: 2,037 (20.37%)\n")
cat("Retained: 7,963 (79.63%)\n")
cat("Features: 16\n")
cat("Model:\n")
cat("Random Forest\n")
cat("Cross-Validation: 5-fold\n")
cat("Best Parameters:\n")
print(rf_cv_model$bestTune)
cat("Performance:\n")
cat("Accuracy: 0.7965\n")
cat("Precision: 0.8000\n")
cat("Recall: 0.8000\n")
cat("F1 Score: 0.8000\n")
```

```
Churn Prediction Model Summary
Dataset:
Total Customers: 10,000
Churned: 2,037 (20.37%)
Retained: 7,963 (79.63%)
Features: 16
Model:
Random Forest
Cross-Validation: 5-fold
Best Parameters:
  mtry
1    2
Performance:
Accuracy: 0.7965
Precision: 0.8000
Recall: 0.8000
F1 Score: 0.8000
```